



¿Es Musubi producible?

Criterio profesional de 6 lentes, unificado, aplicado como vara sobre el código real. Cada hallazgo verificado adversarialmente contra el repo.

72
AGENTES

2.62M
TOKENS

6
LENTES

57
HALLAZGOS VERIFICADOS

1 / 10 / 7 / 8
CRIT / ALTO / MEDIO / BAJO

VEREDICTO EJECUTIVO

Cimientos de nivel 4, sin la espina operacional que el producto exige.

3
/ 5

“Funcional-plus para un autor con sus máquinas”, todavía NO “producible para un equipo o un servicio 24/7”. La distancia a producción es de ~1 nivel, concentrada y bien localizada.

Arquitectura
3 • cimientos sólidos

Dominio IA/SOTA
3 • promesa apagada

Seguridad
3 • sin tenancy

Ingeniería
3 • release débil

Producto/DX
3 • atado al autor

Datos/escala
3 • sin validar

El patrón es geográfico y revelador: donde la memoria *converge* — el nodo central de agregación sobre tailnet — es donde el sistema es más frágil y donde la vara de producción es más alta, mientras el borde local está casi listo. Eso es alentador: los cimientos correctos son lo difícil y ya están hechos (outbox transaccional idempotente, claim con lease auto-recuperado, migraciones en tx con rollback, transporte fail-closed, olvido Ebbinghaus, RRF de 6 señales, redacción RE2+entropía sin deps, CI con -race/coverage-floor). Lo que falta es **disciplina operacional** y una **capa de tenancy/identidad** — construible en semanas, sin reescrituras.

Cinco cosas — ninguna requiere reescritura.

1 • OPERABILIDAD

Sin release ni DR del central

Todo C1–C4+P2 en main sin taggear; perder la `memory.db` central borra toda la memoria del equipo — sin backup off-host ni restore probado.

2 • EQUIPO

Sin modelo multi-tenant

Un único bearer sin identidad/rotación/audit, y cross-project bleed: el recall no filtra por `project_id` y el central descarta la atribución de origen.

3 • MEMORIA

La promesa semántica, apagada

Recall semántico off por default y lo auto-capturado (commits, error→fix) es invisible al vector — sin ningún harness que mida relevancia.

4 • ESCALA

Correctitud sin validar

`/metrics` no expone la salud del pipeline, no hay cuota de crecimiento, y el índice IVF construido para >10k jamás se benchmarkea.

5 • CONFIANZA

El borde del central es implícito

`scope=local` mete un secreto crudo al pozo compartido (la redacción depende del scope declarado por el cliente, no del hecho de que el central ES infra compartida), y no hay threat model documentado.

TENSIONES DE DISEÑO

Los trade-offs que no se resuelven subiendo el modelo.

Model-free vs. promesa de memoria semántica

El determinismo auditable y offline es una fortaleza real y poco común, pero techa la calidad de retrieval y delega el juicio al agente. No se resuelve subiendo el modelo: se resuelve *encendiendo* el recall semántico por default y *midiéndolo*.

Todo-shared vs. privacidad / multi-tenant

El pozo único compartido asume 1 dev / 1 dominio de confianza, pero el objetivo es equipo. El mismo diseño que da “recall federado” produce cross-project bleed. No se puede tener la simplicidad del pozo único Y las fronteras que un equipo exige sin construir tenancy explícita.

Offline-first vs. central como SPOF que necesita DR/HA

El outbox resiliente del cliente *enmascara* la fragilidad del centro. Por ser el único punto de convergencia, el central es el SPOF que MÁS necesita backup/restore — y el que menos tiene.

Simplicidad operacional vs. least-privilege

Apoyar la confidencialidad en WireGuard + un token es lo que hace el onboarding un one-liner; un cerebro de equipo exige credencial por-miembro revocable, authz por operación y audit. Cada capa de identidad erosiona el “pegalo y anda”.

HALLAZGOS VERIFICADOS • 26 RANKEADOS

La distancia a producción, con evidencia.

● 1 crítico ● 10 alto ● 7 medio ● 8 bajo

CRITICAL DR / continuidad del central

esfuerzo M

El cerebro central (SPOF de toda la memoria compartida) no tiene DR: sin backup programado, off-host ni restore probado.

`install-musubi-brain.sh` no configura ningún backup; el único es `backupDB()` `doctor.go:169-197`, que copia al MISMO disco y solo lo dispara el auto-heal. Perder el disco = perder DB y “backup” juntos, irreversible.

→ `systemd .timer` con `VACUUM INTO off-host` (o Litestream WAL-shipping/PITR) + retención + runbook de restore probado end-to-end.

HIGH Release / supply-chain

esfuerzo M

No hay release cortada; la versión tiene dos fuentes de verdad divergentes (0.57 vs v0.77) y los artefactos salen sin firma/SBOM.

HEAD = v0.77.0 + 27 commits sin taggear; `versioninfo.json` congelado en 0.57.0.0; `release.yml` solo emite sha256, sin cosign/SBOM/provenance.

→ Cortar el tag; unificar la versión derivándola del tag; automatizar con goreleaser (cosign keyless + SBOM + SLSA provenance + CHANGELOG).

HIGH Multi-tenancy

esfuerzo L

Cross-project bleed: el recall no filtra por `project_id` y el central descarta la atribución de origen al ingerir.

El predicado de visibilidad (`scope.go:32`) es solo `archived=0 AND superseded_by IS NULL`, usado por TODOS los paths de lectura. El handler ni deserializa `project_id` (`methods.go:111-118`); estampa el del central (`operations.go:207`). Un token lee la memoria shared de TODOS los proyectos.

→ Derivar `project_id` de la credencial; persistir el de origen; filtro a nivel de query en todos los reads; tests de aislamiento A-no-ve-B.

HIGH Borde de confianza

esfuerzo M

Un secreto crudo entra al pozo compartido enviando scope=local al central: la redacción depende del scope declarado, no de que el server ES infra compartida.

`saveObservation` solo redacta si `scope==shared` (`operations.go:173-182`); con `scope=local` persiste crudo. El central usa el mismo handler que el daemon y acepta 'local'.

→ Modo server-side ForceRedact (on cuando el bind es no-loopback): en el ingest del central, redactar SIEMPRE o rechazar `scope=local`.

HIGH Identidad / authz

esfuerzo L

Un único bearer compartido: sin identidad por-principal, sin rotación/revocación selectiva, sin authz por operación ni audit trail.

`http.go:43-44,150-175` resuelve UN token; cualquier portador invoca toda la API mutante; los onboarders persisten el mismo token en cada máquina. Fuga = compromiso total sin revocación selectiva.

→ Credencial por-miembro revocable/rotable; authz mínima por operación (lectura vs escritura); audit log append-only (principal, op, obs_id, ts).

HIGH Promesa central de memoria

esfuerzo M

El recall semántico está APAGADO por default: la memoria out-of-the-box es 100% léxica y medio sistema (embeddings/IVF/RRF) es código latente.

`config.go:403` fija `Provider:'none'`; `static.go:37-40` exige tabla bring-your-own sin embebido ni auto-descarga. Contra Mem0/Zep/Letta que shippean recall vectorial por default, Musubi entrega solo FTS.

→ Embeber (o auto-descargar con checksum) una tabla POTION/model2vec chica; `Provider:'static'` como default, 'none' como opt-out.

HIGH Medición de calidad

esfuerzo M

La relevancia del recall NUNCA se mide: no hay dataset etiquetado, ni métrica, ni gate de regresión.

Cero nDCG/precision@/MRR/LOCOMO/LongMemEval en el repo. Solo microbenchmarks de latencia. Encender el default semántico a ciegas puede REGRESAR la relevancia (embeddings estáticos mean-pooled pueden no superar al FTS en el corpus real).

→ Golden-set etiquetado propio (o subset LongMemEval/LOCOMO) con nDCG@k/precision@k y gate en CI, ANTES de flippear el default.

HIGH Observabilidad

esfuerzo M

/metrics solo cuenta requests HTTP; la salud del pipeline (outbox lag, DB size, drain, p95 recall) no es scrapeable ni alertable.

`observability.go:45-52` emite únicamente `musubi_http_requests_total`. La salud rica del outbox existe en `OutboxHealth()` `outbox.go:255-267` pero solo como tool MCP, no en `/metrics`.

→ Exponer `OutboxHealth` + DB size + p95 recall como gauges Prometheus; alerta de lag sobre `oldest_pending_age`. El struct ya existe: es cablear el render.

HIGH CI / plataforma

esfuerzo M

CI corre solo en ubuntu-latest para un producto Windows-first: los code paths de Windows nunca se ejecutan en el gate de PR.

Los 3 jobs de `ci.yml` en `ubuntu-latest` sin matrix. El producto es Windows-first (`console_windows.go` , `.syso` , `firewall netsh`). Un error en el borde Windows pasa el gate sin detectarse.

→ `strategy.matrix` con `windows-latest` y `macos-latest`; como mínimo build+test en Windows.

HIGH RAM a escala

esfuerzo L

Consolidate y las búsquedas exactas materializan TODO el corpus vivo en RAM por corrida.

`consolidate.go:40-55` hace SELECT sin LIMIT/keyset y carga todo en RAM; `searchExactFullScan` y `loadActiveVectors` escanean todos los vectores sin paginar. A 100k-1M es un pico proporcional al corpus en cada mantenimiento.

→ Pagar Consolidate por bloques (o índice de trigramas en disco/streaming); acotar `loadActiveVectors` por lotes; medir B/op a $n=100k$.

HIGH Validación a escala

esfuerzo M

El índice IVF (régimen >10k para el que existe) prácticamente no se benchmarkea: los benches topan justo en el umbral.

`ExactThreshold=10000` pero los benches iteran $n \in \{1000, 10000\}$; a 10000 el train corre en background sin que el bench espere `Trained()`. Cero benches a 100k/1M.

→ Benches a $n=100k$ (sampled 1M) que fuercen el path IVF entrenado, midiendo p50/p95 y recall@k vs full-scan; gate de regresión de latencia.

MEDIUM Cobertura de redacción

esfuerzo M

La redacción está por debajo de gitleaks/trufflehog: ~8 formas, hex excluido por diseño, familias enteras sin cubrir y allowlist evadible.

El catch-all de entropía no pega hex puro (64 hex → finds=0); connection strings `postgres://user:pass@` pasan; `isAllowlisted` matchea sobre el VALOR (un secreto con 'example' se evade). Sin fuzz.

→ Detectores de alta señal (Slack/GCP-SA/DSN/PII/hex típico), endurecer allowlist a contexto, fuzzear internal/redact, documentar qué cubre y qué no.

MEDIUM Crecimiento / cuota

esfuerzo M

No existe cuota ni tope por proyecto; lo protegido por importancia nunca se archiva ni purga, y las filas 'sent' del outbox nunca se limpian.

`decay.go` saltea el archivado de `importance>=Protect` ; `MarkOutboxSent` solo cambia status (sin DELETE, sin FK cascade). El decay acota el recall pero NO el footprint.

→ Cuota (filas/bytes) por proyecto con expulsión LRU; purga de outbox 'sent'/huérfanas en Maintain; uso vs cuota en /metrics.

MEDIUM Paridad de embeddings

esfuerzo M

C3 (commits) y C4 (error→fix) se guardan con embedding nil: invisibles al vector aun con provider configurado.

`capture.go:180` y `methods.go:1416` pasan nil; `buildTurnRecall` nunca setea `QueryVector` . La ruta manual Sí embebe — solo las automáticas (el diferenciador del track) quedan fuera.

→ Dar embedder al proceso de turn/capture (o rutear al daemon); embeber C3/C4 y el prompt; backfill de lo capturado con nil.

MEDIUM Contradicciones / staleness

esfuerzo L

La resolución de contradicciones es manual e incompleta; memoria stale/contradictoria puede filtrarse al recall sin marca.

`conflicts.go:100-147` : auto-supersede solo con mismo topic_key + sim≥0.7; similitud media queda `pending` esperando `musubi_judge` . El predicado de visibilidad no excluye 'pending' — dos memorias contradictorias coexisten y ambas se sirven.

→ Demote/anotar en el ranking las observaciones con conflicto 'pending' sin resolver; la detección ya corre, falta que el recall la respete.

MEDIUM Provisioning de cero

esfuerzo L

“Instalar musubi = máquina 100% lista” no se cumple: provision no instala Tailscale y P3 (dev tools)/P4 (repos+git/ssh) están pendientes.

`provision.go:142` devuelve `StatusTodo`; el comando `Go` detecta y puede unir con `--authkey` pero NO instala el binario de la malla; solo el PS1 lo hace. Sin código P3/P4.

→ Que provision instale/una Tailscale él mismo (o falle-cerrado con instrucciones), y completar P3/P4 — o recortar la promesa en la doc.

MEDIUM Continuidad multi-máquina

esfuerzo M

La continuidad PC↔laptop solo cubre scope 'shared': la memoria y auto-captura locales quedan atrapadas en cada disco.

El outbox solo empuja shared; C3/C4 se guardan LOCAL. El scope local nunca sale de la máquina ni hay pull del central. Justo lo que la auto-captura presume como valor nace local y no sigue al usuario.

→ Definir la continuidad del scope local por-usuario (sync por identidad, o pull selectivo) — o documentar que solo shared es portable.

MEDIUM Migraciones / compat

esfuerzo S

Sin guarda de compatibilidad de esquema: un binario viejo opera en silencio sobre una DB migrada más nueva.

`applyMigrations` solo aplica `version>current`; si `current` supera lo conocido, es no-op y retorna nil sin negarse.

`latestSchemaVersion()` existe pero solo se usa en tests. Riesgo de corrupción lógica en flota mixta.

→ Al abrir, si `user_version > latestSchemaVersion()`, negarse (o read-only con warning). Fixtures de DB pre-migración en CI.

MEDIUM FTS5 / footprint

esfuerzo M

FTS5 sin external-content duplica el corpus 2x y el trigger AFTER UPDATE re-tokeniza todo el contenido en cada bump de acceso.

`observations_fts` sin `content=` guarda una segunda copia íntegra; el trigger `observations_au` sin `WHEN / UPDATE OF` dispara DELETE+INSERT re-tokenizando en cada update de metadatos.

→ Migrar a FTS5 external-content; fix inmediato: `AFTER UPDATE OF content, topic_key` o `WHEN new.content<>old.content`.

MEDIUM Adopción por terceros

esfuerzo M

Defaults personales del autor hardcoded: IP de tailnet y repoOwner embebidos en el binario distribuible.

`probe.go:15` `DefaultBrain='100.79.126.62:7717'` ; `update.go:14-16` `repoOwner='codeabraham16'` como const sin override (bloqueo duro de fork). Toda la doc en español.

→ Quitar el default de cerebro (exigir `--brain` sin IP embebida); parametrizar `repoOwner` por build var; README/runbooks bilingües.

LOW Threat model / transporte

esfuerzo S

`allow_insecure_token: true` se hornea por default en cada config provisionada y no hay threat model formal.

`synconfig.go:37` escribe `allow_insecure_token: true` (con `http://`) incondicional. La confidencialidad descansa 100% en WireGuard; si el token se reusa fuera del tailnet, viaja claro.

→ Threat model explícito (qué cubre WireGuard y qué no); preferir TLS aun intra-tailnet o no hornear `insecure_token` en silencio.

LOW Superficie HTTP

esfuerzo S

Sin rate-limit ni lockout ante fuerza bruta del bearer; solo se cuenta el 401.

`http.go` rechaza con 401 pero no hay throttle/backoff/lockout. El adivinado online del token es ilimitado para cualquier peer del tailnet.

→ Rate-limit por IP/conexión en el path de auth (o lockout tras N 401) con limiter en memoria; alerta al superar umbral de 'unauthorized'.

LOW Least-privilege de red

esfuerzo S

La regla de firewall abre TODO el rango CGNAT del tailnet (100.64.0.0/10) inbound, sin ACLs de Tailscale.

`network.go:83-86` crea `TS-Allow-Tailnet-In` con `RemoteAddress 100.64.0.0/10`. Sin ACLs de Tailscale, cualquier dispositivo de la malla + el token = movimiento lateral amplio.

→ ACLs de Tailscale (tailnet policy) que limiten el puerto del brain a principals concretos; no confiar solo en el rango CGNAT.

LOW CI / higiene

esfuerzo S

Sin linter de seguridad (gosec/bodyclose), tool @latest no hermético y coverage floor solo global.

`.golangci.yml` usa 'standard' sin gosec sobre red/SQL/secretos; `govulncheck@latest` no pineado; el floor compara solo el total, enmascarando paquetes a 0%.

→ gosec+bodyclose scopeados a red/SQL/secretos; pinear tools; piso de cobertura por-paquete.

LOW Concurrencia / proceso efímero

esfuerzo M

Cada invocación CLI/hook reabre la DB y re-ejecuta backfills en cada open, compitiendo por el escritor WAL.

`NewDbEngine` corre migraciones+backfills en cada open; `BackfillOutbox` es O(n) ungated con write-tx. El hook Stop abre un engine nuevo por turno; a volumen puede pegar `busy_timeout`.

→ Gatear `BackfillOutbox` por flag en meta; para el hook, rutar la captura al daemon vía IPC en vez de abrir la DB en paralelo.

LOW Provisioning / robustez

esfuerzo S

El setup local de provision es best-effort sin reportar el estado parcial en el veredicto.

`injectLocalSetup` es best-effort sin rollback; el switch de veredicto chequea `Connected` primero, así que imprime verde aunque un hook/skill local haya fallado.

→ Reportar setup local parcial en el veredicto cuando un paso local falle; marcar el proyecto como incompleto para reintento idempotente.

LOW Onboarding / DX

esfuerzo M

El onboarding Windows acopla el fix de NordVPN y misdiagnostica cualquier fallo como "falta el split-tunnel".

`connect-brain-windows.ps1` aplica el fix WFP siempre y hardcodea 'falta el split-tunnel' como causa de CUALQUIER fallo. El hermano Linux Sí condiciona a `command -v nordvpn`.

→ Detectar NordVPN y solo entonces aplicar el fix/mensaje; separar el happy-path puro-Tailscale de la nota de convivencia.

LOW Docs / drift

esfuerzo 5

El README documenta 3 hooks pero setup instala 4 (falta el hook Stop/C3) y los enlaces del CHANGELOG están stale.

README.md:173/177 y el diagrama Mermaid listan 3 hooks; setup.go:195-199 escribe un cuarto (Stop/C3). CHANGELOG [Unreleased] apunta a v0.44.0 (33 releases stale).

→ Actualizar README (texto+tabla+diagrama) a los 4 hooks; automatizar los enlaces del CHANGELOG en el script que corta el tag.

LOW Índices / FTS tokenizer

esfuerzo 5

FTS5 sin tokenizer explícito ni prefix index y el hot-path de recencia sin índice temporal.

fts5 sin tokenize= (cae a remove_diacritics=1) ni prefix= mientras el recall emite queries stem*; recentCandidates hace ORDER BY temporal sin índice de tiempo.

→ tokenize='unicode61 remove_diacritics 2' + prefix='2 3' vía migración; índice compuesto de recencia si el fallback crece.

ROADMAP A PRODUCCIÓN • 5 FASES

De “main” a servicio de equipo, en orden de palanca.

FASE 0 • DÍAS, NO SEMANAS

Destrabar

Convertir “main” en un producto con release verificable y eliminar el riesgo existencial de perder toda la memoria compartida.

- Cortar el primer tag: mover CHANGELOG [Unreleased] → [X.Y.Z], taggear, dejar que release.yml publique C1–C4+P2.
- DR real del central: systemd .timer con VACUUM INTO off-host + retención + runbook de restore probado.
- Unificar la versión en una sola fuente derivándola del tag; test de consistencia versioninfo==tag.
- Guarda de compat de esquema en runtime: usar latestSchemaVersion() (ya existe) para negarse ante DB más nueva.

Sin release no hay producto, solo main; sin DR el central es un SPOF cuya pérdida es irreversible. Máximo ROI, mínimo esfuerzo.

Modelo de equipo

Que el “cerebro compartido” sea realmente multi-tenant, revocable y con un borde de confianza explícito en el central.

- Aislamiento por project_id derivado de la credencial: filtrar todos los reads detrás del seam de scope.go; conservar el project_id de origen.
- Identidad por-principal: N tokens con claims (project+rol) revocables; authz mínima por operación; audit log append-only.
- Redacción forzada server-side: flag 'shared-infra' que en el central redacte SIEMPRE o rechace scope=local.
- Threat model documentado + rate-limit/lockout en auth + ACLs de Tailscale; tests de aislamiento A-no-ve-B.

El propósito es EQUIPO, pero el diseño “todo converge a shared” produce bleed y atribución rota. Es donde la vara es más alta y el sistema más débil.

Encender la promesa de memoria

Cumplir el diferenciador declarado (memoria semántica model-free) con evidencia de que no regresa la calidad.

- Harness de calidad de recall PRIMERO: golden-set etiquetado con nDCG@k/precision@k y gate en CI.
- Embeber (o auto-descargar con checksum) tabla POTION/model2vec; Provider:'static' como default.
- Paridad de embeddings en TODA la captura: embeber C3/C4 y el prompt; backfill de lo capturado con nil.
- Demote/anotar en el ranking las contradicciones 'pending' para no servir memoria stale sin marca.

Hoy medio sistema es código latente. Encender a ciegas puede regresar la relevancia: el harness va PRIMERO — medir antes de fliepear.

Operabilidad a escala

Volver el servicio observable, acotado en crecimiento y validado en el régimen para el que fue diseñado.

- Observabilidad accionable en /metrics: OutboxHealth + DB size + p95 recall como gauges, con alerta de lag.
- Crecimiento acotado: cuota por project_id con expulsión LRU, purga de outbox 'sent'/huérfanas.
- Validación a escala: benches IVF a n=100k (sampled 1M) con recall@k vs full-scan y gate de perf; acotar RAM de Consolidate.
- FTS5 external-content (elimina 2x) + trigger restringido a UPDATE OF content.

Un servicio 24/7 necesita ver su salud, no crecer sin techo y probar que rinde a volumen. Estos gaps no rompen hoy — rompen a volumen de equipo real.

Endurecimiento y adopción por terceros

Que un tercero o un teammate pueda instalar, operar y forkear sin la máquina ni la infra del autor.

- CI matriz Windows+macOS + gosec/bodyclose sobre red/SQL/secretos + tools pineadas + coverage por-paquete.
- Quitar defaults personales: DefaultBrain sin IP embebida, repoOwner parametrizable por build var.
- Docs bilingües con runbooks completos (restore/upgrade/rotación de token) + fix docs 3→4 hooks + CHANGELOG automatizado.
- Completar P3 (dev tools) y P4 (repos+git/ssh) o recortar la promesa; reportar setup local parcial; detectar NordVPN antes del fix.
- Ampliar cobertura de redacción a paridad gitleaks (Slack/GCP-SA/DSN/hex/PII) + fuzzing sobre internal/redact.

Los cimientos correctos ya están; lo que falta para "producible" es que Musubi deje de estar atado al autor.

Lo que rinde apenas lo tocás.

- 1 Cortar el tag de release: un commit destraba la cadena instalador→versión y entrega C1–C4+P2 que hoy están solo en main.
- 2 Guarda de compat de esquema: `latestSchemaVersion()` ya existe, solo hay que invocarla en el open (esfuerzo S, previene corrupción en flota mixta).
- 3 Forzar redacción / rechazar scope=local en el central vía flag: cierra el hueco de secreto crudo sin tocar el borde local.
- 4 Exponer OutboxHealth + DB size en /metrics: el struct ya existe, es cablear el render — primera alerta de lag del sync.
- 5 Purgar filas outbox 'sent'/huérfanas en Maintain: corta el crecimiento monótono de la tabla de sync.
- 6 Restringir el trigger FTS a `AFTER UPDATE OF content`: elimina la re-tokenización masiva con un cambio de una línea.
- 7 Añadir windows-latest a la matriz de CI: el código Windows-first dejaría de mergearse sin ejecutarse jamás.
- 8 Rate-limit/lockout tras N 401 en auth: limiter en memoria barato contra fuerza bruta sobre un servicio alcanzable por todo el tailnet.
- 9 Quitar los defaults personales hardcodeados (IP, repoOwner): un tercero deja de apuntar a la infra del autor por default.
- 10 Fix docs 3→4 hooks + enlaces stale del CHANGELOG: restaura la confianza en la doc que un adoptante audita.